Day 4 – Routing & Navigation

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Dev Server Running**: Ensure `ng serve` is active in the terminal.
   - **IDE Ready**: Open your project in VS Code. Prepare to create a routing file and generate new routing components.

2. **Prerequisites**:
   - You must have completed Day 3 successfully (Encapsulating data inside a `GameService` and subscribing to it in components).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain how Angular Router handles client-side navigation in a Single Page Application (SPA).
- Define route patterns and link them to components in a routing configuration file.
- Use the `<router-outlet>` directive to render routed component views dynamically.
- Implement component navigation links using `routerLink` and style active links with `routerLinkActive`.
- Configure dynamic routes using path parameters (`:id`) and read parameters using `ActivatedRoute`.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define routing in SPAs. Compare browser-reloads with client-side DOM swapping.
2. Key Concepts & Core Ideas	7 min	Explain Route definitions, RouterOutlets, RouterLinks, and ActivatedRoute parameters.
3. Live Coding Walkthrough	23 min	Configure routing files, create detail pages, retrieve dynamic ID path parameters, and wire navigation headers.
4. Check for Understanding	5 min	Q&A on route configurations, route order priority, and parameter subscriptions vs snapshots.

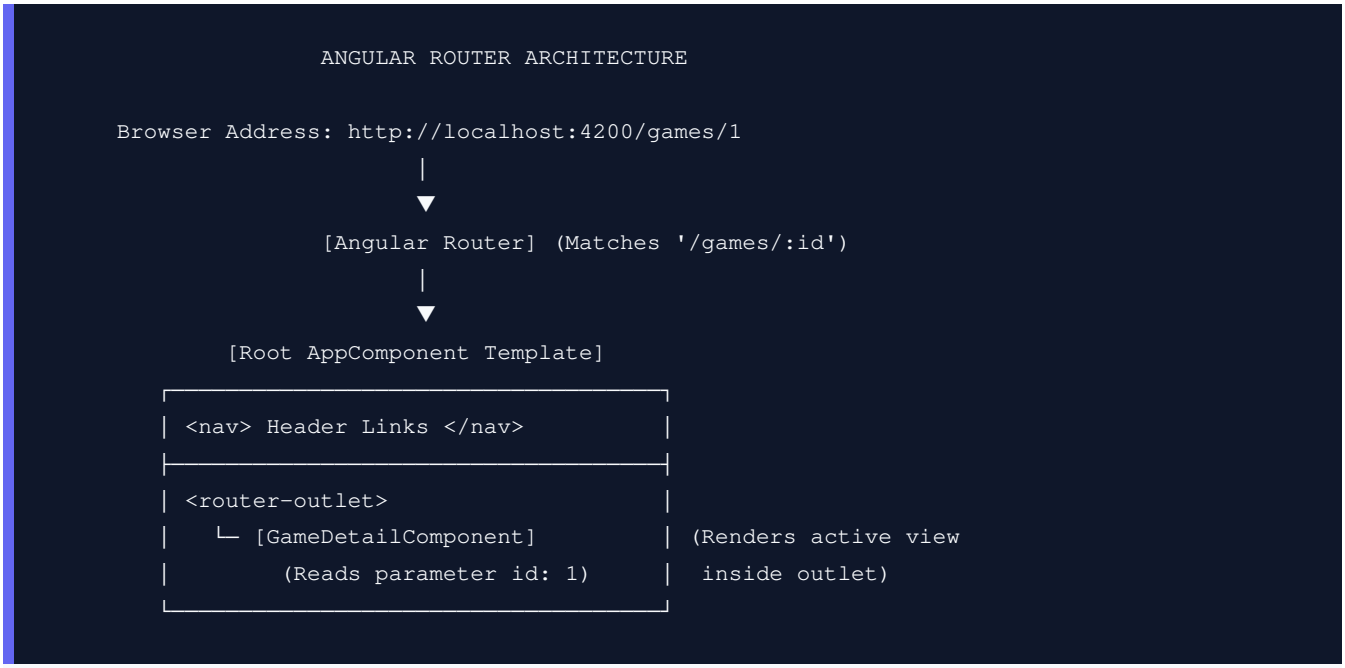
5. Class Wrap-up & Checkpoint	3 min	Verify navigation links and dynamic page loading operate correctly in the browser.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

In standard web applications, clicking a link triggers a full page refresh: the browser sends an HTTP request to the server, receives a new HTML document, and repaints the screen. In Angular SPAs, clicking a link is intercepted. Angular's **Router** checks the configuration, updates the URL address in the browser bar (without reloading the page), and swaps the active component in the DOM.

- **Navigation Flow**: We want to add navigation so clicking a game in the catalog navigates the user to a detailed view of that specific game (`/games/1`), loading a `GameDetailComponent` dynamically.



2. Key Concepts & Core Ideas (7 min)

Introduce these routing structures:

- **`app.routes.ts`**: The configuration file where you declare a list of Route objects containing paths and target components.
- **`<router-outlet>`**: A placeholder directive that acts as a viewport. Angular Router renders the component matched by the active route inside this placeholder.
- **`routerLink`**: Directive used on HTML anchor tags (`<a>`) instead of `href` to tell Angular to handle navigation client-side, preventing browser refreshes.
- **`routerLinkActive`**: Directive that applies CSS class stylings to navigation links only when their route

matches the browser URL.

- **Dynamic Route (`path: 'games/:id'`)**: A route containing a variable path parameter (`:id`).
- **`ActivatedRoute`**: An injectable class that contains metadata about the active route, allowing you to read path parameters from the URL stream.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Configuring the Router Routes

- **Concept**: We will define routes in `src/app/app.routes.ts` mapping paths to components.
- **Code**: Create `src/app/app.routes.ts`:

```
import { Routes } from '@angular/router';
import { GameListComponent } from './components/game-list/game-list.component';
import { GameDetailComponent } from './components/game-detail/game-detail.component';

export const routes: Routes = [
  # Default route: redirects empty path to the catalog list page
  { path: '', redirectTo: 'games', pathMatch: 'full' },

  # Standard catalog listing route
  { path: 'games', component: GameListComponent },

  # Dynamic parameter route: matches paths like /games/1 or /games/abc
  { path: 'games/:id', component: GameDetailComponent },

  # Wildcard fallback route: matches any invalid URL and handles 404s
  { path: '**', redirectTo: 'games' }
];
```

Ensure routing is initialized globally in `src/app/app.config.ts`:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    # Register the routes provider globally in the DI container
    provideRouter(routes)
  ]
};
```

Step 3.2: Creating the Game Detail Component

- **Concept**: Generate the detailed view component. We will inject `ActivatedRoute` to read the ID, and use

to look up the game's details.

- **Code**: Generate the component:

```
ng generate component components/game-detail
```

Modify `src/app/components/game-detail/game-detail.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, RouterModule } from '@angular/router';
import { GameService, Game } from '../../services/game.service';

@Component({
  selector: 'app-game-detail',
  standalone: true,
  imports: [CommonModule, RouterModule],
  templateUrl: './game-detail.component.html',
  styleUrls: ['./game-detail.component.css']
})
export class GameDetailComponent implements OnInit {
  game: Game | undefined;

  // Inject ActivatedRoute to inspect URL details, and GameService to lookup inventory
  constructor(
    private route: ActivatedRoute,
    private gameService: GameService
  ) {}

  ngOnInit(): void {
    # Read the dynamic 'id' parameter from the route parameter map
    # The snapshot provides a simple one-time read.
    const gameId = Number(this.route.snapshot.paramMap.get('id'));

    this.gameService.getGames().subscribe({
      next: (games: Game[]) => {
        # Look up the game matching the parsed ID
        this.game = games.find(g => g.id === gameId);
      }
    });
  }
}
```

Modify `src/app/components/game-detail/game-detail.component.html`:

```
<div class="detail-container">
  @if (game) {
    <h2>🎮 Game Details: {{ game.title }}</h2>
    <div class="card">
      <p><strong>ID:</strong> {{ game.id }}</p>
      <p><strong>Price:</strong> ${{ game.price | number:'1.2-2' }}</p>
      <p><strong>Status:</strong>
        <span class="status" [ngClass]="game.available ? 'available' : 'out-of-stock'">
```

```
          {{ game.available ? 'In Stock' : 'Out of Stock' }}
        </span>
      </p>
    </div>
  } @else {
    <p class="error-msg">Game not found.</p>
  }

  <!-- routerLink handles navigation back to the catalog without reloading -->
  <a routerLink="/games" class="btn-back">← Back to Catalog</a>
</div>
```

Step 3.3: Updating Root Layout and Navigation Links

- **Concept**: Update the catalog list component to show navigation links, and configure `AppComponent` to use `<router-outlet>`.
- **Code**: Modify `src/app/components/game-list/game-list.component.html` (inside the `@for` loop of games, add a link to details):

```
...
<li class="game-item">
  <span class="title">{{ game.title }}</span>
  <span class="price">${{ game.price | number:'1.2-2' }}</span>

  <!-- Dynamic routerLink takes an array: route segment and dynamic parameters -->
  <a [routerLink]="['/games', game.id]" class="btn-link">View Details</a>
  ...
```

Ensure `GameListComponent` imports `RouterModule` in `src/app/components/game-list/game-list.component.ts`:

```
import { RouterModule } from '@angular/router'; // Add this import
...
@Component({
  ...
  imports: [CommonModule, FormsModule, RouterModule], // Register in imports
  ...
```

Modify `src/app/app.component.ts` to import `RouterModule`:

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router'; // Import RouterModule

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterModule], // Replace GameListComponent with RouterModule
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
```

```
})
export class AppComponent {}
```

Modify `src/app/app.component.html`:

```
<header>
  <h1>Welcome to GameKey Store</h1>
  <nav>
    <!-- routerLinkActive applies active-link CSS class if route matches URL -->
    <a routerLink="/games" routerLinkActive="active-link">Catalog Inventory</a>
  </nav>
</header>

<!-- router-outlet is the placeholder viewport for routed components -->
<router-outlet></router-outlet>
```

- **Verify**: Check `http://localhost:4200/`. Clicking "View Details" on a game should swap the viewport with the details card, and the browser URL will update to `/games/1` without refreshing the tab.

---

4. Check for Understanding (5 min)

> ***Question***: *"When should we read route parameters using `route.snapshot` vs subscribing to `route.paramMap`?"*
> ***Answer***: *`route.snapshot` is a simple snapshot read. Use it when the component will never be reloaded with different parameters while already open (e.g. going from catalog to details). Subscribing to `route.paramMap` is required if the component stays active but route parameters change dynamically (e.g., navigating from `/games/1` directly to `/games/2` via a next button on the details page).*

> ***Question***: *"What happens if you define a wildcard route `{ path: '**', component: PageNotFoundComponent }` at the beginning of the routes configuration list?"*
> ***Answer***: *Router paths are matched sequentially in the order defined in the routes list. Since `**` matches everything, placing it at the beginning would intercept all URLs, rendering `PageNotFoundComponent` for every request, blocking other routes. Always place the wildcard route last.*

> ***Question***: *"How does client-side routing prevent backend 404 errors during browser reloads on URLs like `/games/1`?"*
> ***Answer***: *When you refresh `/games/1`, the browser makes a direct request to the server for that path. Unless the server (e.g. Nginx, Apache) is configured to redirect all fallback URLs to `index.html`, the backend server returns a standard 404 error. The backend server must serve `index.html` for all*

> *unknown routes, allowing Angular Router to intercept and handle the URL path client-side.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ Clicking "View Details" navigates to `/games/:id` and updates the URL without browser refreshes.
  - ○ The `GameDetailComponent` retrieves the dynamic ID parameter from the URL snapshot.
  - ○ Clicking the back link successfully returns the user to the catalog inventory view.